

Python-клиент Redis (или KeyDB)

Подключение к Redis

Клиенты Redis для ускорения повторных вызовов используют пул соединений с сервером, работающий по "ленивой" схеме, при которой реальное подключение к серверу производится при выполнении первой команды по соединению.

По умолчанию при подключении для аутентификации на кластере Redis клиент использует пользовательскую роль `default` из описания кластера, если же кластер не защищён подключение производится без аутентификации. При необходимости можно указать другую пользовательскую роль для аутентификации на кластере.

Подключение к кластеру Redis

Типичный сценарий - подключение по имени кластера (названию параметра из конфигурации):

```
1 client = sbis.RedisClient.FromClusterName('Модуль.Хранилище')
2
3 # С аутенфикацией под пользовательской ролью "admin".
4 client = sbis.RedisClient.FromClusterName('Модуль.Хранилище', 'admin')
```

Подключение с передачей структуры [ClusterEndpoint](#):

```
1 endpoint =
2     sbis.Config.Instance().GetOrThrowClusterEndpoint('Модуль.Хранилище')
3 client = sbis.RedisClient(endpoint, 'Модуль.Хранилище')
4
5 # С аутенфикацией под пользовательской ролью "admin".
6 client = sbis.RedisClient(endpoint, 'Модуль.Хранилище',
7     user_role='admin')
```

Прямое подключение к узлу кластера Redis

При необходимости можно подключиться к конкретному узлу кластера Redis с использованием структуры [RedisEndpoint](#):

```
1 # По имени кластера (названию параметра из конфигурации).
2 endpoint = sbis.RedisEndpoint.FromClusterName('Модуль.Хранилище',
3     'slave1')
4 client = sbis.RedisClient(endpoint)
5
6 # С аутенфикацией под пользовательской ролью "admin".
7 endpoint = sbis.RedisEndpoint.FromClusterName('Модуль.Хранилище',
8     'slave1', 'admin')
9 client = sbis.RedisClient(endpoint)
10
11 # По точке подключения к кластеру.
12 cluster_endpoint =
13     sbis.Config.Instance().GetOrThrowClusterEndpoint('Модуль.Хранилище')
14 endpoint = sbis.RedisEndpoint.FromClusterEndpoint(cluster_endpoint,
15     'Модуль.Хранилище',
16     'slave1')
17 client = sbis.RedisClient(endpoint)
18
19 # С аутенфикацией под пользовательской ролью "admin".
```

```

18 endpoint = sbis.RedisEndpoint.FromClusterEndpoint(cluster_endpoint,
19                                                     'Модуль.Хранилище',
20                                                     'slave1',
21                                                     'admin')
22 client = sbis.RedisClient(endpoint)

```

Прямое подключение к серверу Redis

Для целей локального тестирования можно воспользоваться подключением к серверу Redis по имени узла или IP-адресу с использованием структуры [RedisEndpoint](#):

```

1 endpoint = sbis.RedisEndpoint()
2 endpoint.host = '127.0.0.1'
3 endpoint.port = 4321
4 client = sbis.RedisClient(endpoint)
5
6 # С аутентификацией под пользователем "user" с паролем "pass".
7 endpoint = sbis.RedisEndpoint()
8 endpoint.host = '127.0.0.1'
9 endpoint.port = 4321
10 endpoint.username = 'user'
11 endpoint.password = 'pass'
12 client = sbis.RedisClient(endpoint)

```

Подключение к кластеру Redis с использованием строки подключения



Не рекомендовано!

При использовании строки подключения клиент не получает полных сведений о кластере Redis, к одному из серверов которого подключается, и не может скорректировать логику взаимодействия с сервером.

Как следствие, в некоторых редких сценариях могут возникать ошибки подключения, и даже возможно подключение к серверам в других кластерах.

```

1 client = sbis.RedisClient.FromConnectionString("host='10.10.10.10'
2 port='1234' dbindex='2'")
3
4 # С аутентификацией под пользователем "user" с паролем "pass".
5 client = sbis.RedisClient.FromConnectionString("host='10.10.10.10'
6 port='1234' dbindex='2'",
7                                                  'user', 'pass')

```

Поля `port` (по умолчанию — 6379) и `dbindex` (по умолчанию — 0) являются необязательными.

Команды Redis

Команды, поддерживаемые клиентом для Python, реализованы методами класса [RedisClient](#). Названия команд и их аргументов соответствуют официальным именам из документации Redis. Описание команд можно найти в документации Redis: [Command reference – Redis](#).

Аргументы команд

Строковые параметры команд могут задаваться аргументами как типа `str` (юникод-строкой), так и `bytes` (байтовые-строки):

```

# Как значения.
client.Set('key', 'value')
client.Set('key', b'value')

```

```
# Так и ключи.
client.Set(b'key', 'value')
client.Set(b'key', b'value')
```

Аналогично и с параметрами, принимающими списки, множества и словари, но с ограничением: все их элементы должны быть одного типа:

```
# Списки.
client.Del(['key1', 'key2'])
client.Del([b'key1', b'key2'])
# Множества.
client.SMIsMember('key', {'member1', 'member2'})
client.SMIsMember('key', {b'member1', b'member2'})
# Словари.
client.HMSet('key', {'field1': 'value1', 'field2': 'value2'})
client.HMSet('key', {'field1': b'value1', 'field2': b'value2'})
client.HMSet('key', {b'field1': 'value1', b'field2': 'value2'})
client.HMSet('key', {b'field1': b'value1', b'field2': b'value2'})
```

При этом аргументы типа `str` при передаче на сервер Redis преобразуются в соответствии с установленной клиенту кодировкой (по умолчанию — `CP1251`).

Возвращаемые значения

Таблица трансляции типов данных Redis в Python типы SBIS:

- Простые строки (статусы успеха) → `None`.
- Ошибки (статусы неудачи) → `RedisError`.
- Целые числа → `int`.
- Целые числа, ограниченные значениями 0 и 1 → `bool`.
- Объемные строки → `str`, `bytes`.
- Массивы → `list[str]`, `list[bytes]`, `list[int]`.
- Массивы уникальных значений → `set[str]`, `set[bytes]`.
- Массивы пар ключ-значение → `dict[str, str]`, `dict[str, bytes]`, `dict[bytes, bytes]`.

Классы команд, результат которых – объёмная строка или массив, в дополнение к основному методу реализуются методом с суффиксом `Bin`, возвращающим байт-строку вместо юникод-строки:

```
1 # Без суффикса возвращаются str.
2 elems: set[str] = client.SPop('key', 3)
3
4 # С суффиксом Bin -- bytes.
5 elems: set[bytes] = client.SPopBin('key', 3)
```

Однако методы с суффиксом `Bin` меняют только возвращаемый тип значений ключей и полей и элементов множеств, а имена ключей и полей, как и методы без суффикса, возвращают в юникод-строках.

```
1 # Без суффикса возвращаются str.
2 kv: dict[str, str] = client.HGetAll('key')
3
4 # С суффиксом Bin -- bytes только в значениях.
5 kv: dict[str, bytes] = client.HGetAllBin('key')
```

Для некоторых команд реализованы специальные типы возвращаемых значений. Например, команда `Eval` возвращает произвольный тип, соответствующий результату скрипта Lua.

Исполнение одиночных команд

Посылка команды

```
value: str = client.Get('key')
```

В качестве результата возвращается объект типа, соответствующего результату команды Redis, транслированному в соответствии с указанной выше таблицей.

Исполнение последовательностей команд

Python-клиент поддерживает посылку последовательностей команд на сервер. Это позволяет оптимизировать общение с сервером за счёт упразднения сетевых задержек между отдельными командами.

Посылка команд

Последовательность формируется путем создания объекта последовательности [RedisCommandBatch](#) и добавления в него команд с соответствующими аргументами. Затем последовательность передаётся клиенту на исполнение. Порядок добавления команд определяет порядок их исполнения на сервере.

```
1 batch = RedisCommandBatch()
2 batch.Get('key') \
3     .Set('key', 'value') \
4     .Time()
5 client.SendCommandBatch(batch)
```

Извлечение результата

Результаты команд могут быть извлечены из объекта последовательности:

```
1 value: str = batch.GetResult(0)
2 time: datetime.datetime = batch.GetResult(2)
```

Атомарное исполнение

Redis поддерживает атомарное исполнение последовательностей команд, называемых Redis-транзакцией. Транзакция гарантирует, что никакой запрос от других клиентов не может вклиниться между командами транзакции, и что либо будут исполнены все команды транзакции, либо ни одной. Подробное описание транзакций можно найти в официальной документации: [Transactions – Redis](#).

Формируются транзакции так же, как и обычные последовательности:

```
1 batch = RedisCommandBatch()
2 batch.Get('key') \
3     .Set('key', 'value') \
4     .Time()
```

Сформированная транзакция передается на исполнение клиенту. В качестве статуса исполнения транзакции клиент выдает значение типа bool: **True**, если транзакция исполнена, **False**, если отменена по изменению отслеживаемых ключей:

```
1 executed : bool = client.SendAtomicCommandBatch(batch);
2 if executed:
3     value: str = batch.GetResult(0)
4     time: datetime.datetime = batch.GetResult(2)
```

Попытка извлечь результаты команд неисполненной транзакции приведёт к выбросу исключения.

Настройки Python-клиента

Python-клиент Redis позволяет изменять некоторые аспекты своего поведения:

1. Конфиденциальность (`confidential`) – позволяет скрывать аргументы команд в логах. По умолчанию аргументы логируются.
2. Кодировка (`encoding`) – позволяет устанавливать кодировку, используемую для хранения строк. По умолчанию используется кодировка `CP1251`.
3. Чтение с мастера (`read_from_master`) – позволяет перенаправлять команды чтения на мастер кластера Redis. По умолчанию чтение данных выполняется с реплик.
4. Таймаут (`timeout`) – позволяет установить предельное время исполнения команды. По умолчанию используется таймаут кластера, если он не выставлен – 30 секунд.
5. Серьёзность ошибок (`error_severity`) – позволяет изменять тип сообщений, выводимых в лог при проблемах доступа к БД сервера Redis. По умолчанию фатальными считаются ошибки логики приложения, некорректные параметры подключения, сетевые ошибки и истечение таймаута, остальные логируются как простые ошибки.

Настройки можно выставить при создании клиента или с помощью свойства `settings`:

```
1 # Понижаем серьёзность сетевых проблем до предупреждений, а таймаутов -  
  # до некритических  
2 # ошибок, для всех команд, направляемых через инициализируемый  
  экземпляр клиента.  
3 settings = RedisSettings(error_severity={  
4     RedisErrorTypes.NETWORK: RedisErrorSeverity.Warning  
5     RedisErrorTypes.TIMEOUT: RedisErrorSeverity.Error  
6 })  
7 client = sbis.RedisClient.FromClusterName('Модуль.Хранилище',  
  settings=settings)  
8  
9 # Устанавливаем кодировку UTF8 и чтение с мастера для всех последующих  
  команд с клиента.  
10 settings = RedisSettings(encoding=EncodingType.UTF8,  
  read_from_master=True)  
11 client.SetSettings(settings)
```

Во всех вариантах невыставленные настройки остаются при своих текущих значениях.

В дополнение инициализаторы Python-клиента Redis предоставляют ещё один параметр - модификатор поведения:

- Декодировать ответы (`decode_responses`) — позволяет отключать декодирование строковых результатов исполнения команд Redis. При установке параметра в значение `False` все методы клиента будут возвращать исключительно байт-строки, не предпринимая попыток их декодировать в соответствии с настройкой `encoding`. По умолчанию ответы декодируются.

Особенности работы с асинхронным клиентом Redis



Python-клиент Redis не потокобезопасен!

Обращаться к экземпляру клиента из разных потоков недопустимо. В лучшем случае при одновременном обращении будет выброшено исключение с текстом *"Нельзя посылать новые команды до завершения текущей асинхронной команды!"*, либо процесс сервиса завершится с падением, а в худшем будут испорчены данные на сервере Redis.

Клиенты Redis используют общий пул соединений с кластерами Redis, поэтому кэширование экземпляров клиента Redis в прикладном коде не рекомендуется.